T8 Perceptrones multicapa

Índice

1. Introducción
2. Perceptrón: el problema XOR
3. Perceptrones multicapa
4. Funciones de activación
5. TensorFlow Playground
6. Ejemplo de MLP para clasificación
7. Ejemplo de MLP para regresión
8. Consideraciones finales

1 Introducción

Objetivo: introducir los perceptrones multicapa con ejemplos de uso

El problema XOR: problema simple que no puede resolverse con un modelo lineal clásico (Perceptrón)

Perceptrones multicapa: generalización del Perceptrón clásico que constituye la base de las redes actuales

TensorFlow Playground: herramienta web para jugar con redes sencillas aplicadas a datos 2d

Ejemplo de MLP para clasificación: tarea MNIST

Ejemplo de MLP para regresión: tarea auto-mpg

Consideraciones finales: antecedentes, importancia de la profundidad, etc.

2 Perceptrón: el problema XOR

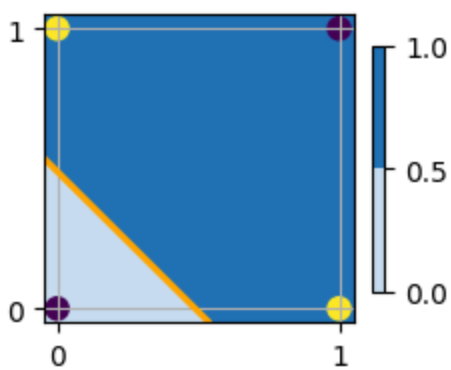
Perceptrón: versión determinista de regresión logística binaria

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \mathbb{I}(\boldsymbol{w}^t\boldsymbol{x} + b \geq 0) = H(\boldsymbol{w}^t\boldsymbol{x} + b)$$

Libro Perceptrons: publicado en 1969 por Marvin Minsky y Seymour Papert, proporcionó numerosos ejemplos de problemas de clasificación que los perceptrones no resuelven

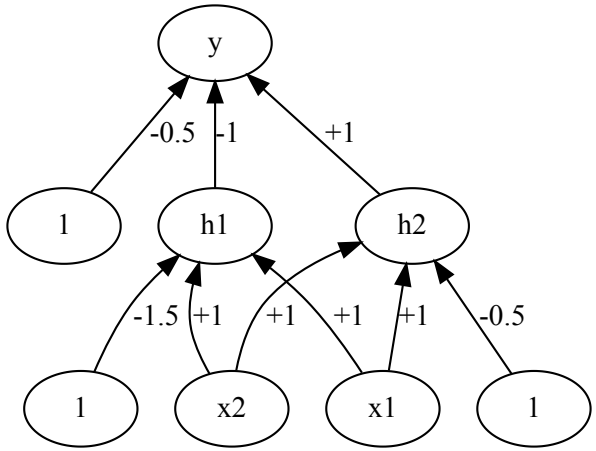
Problema XOR: problema famoso del libro Perceptrons con datos 2d no linealmente separables

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([0, 1, 1, 0], dtype=int); # print(np.c_[X, y])
w = np.array([1, 1]); b = -0.5; P = lambda x: np.heaviside(w @ x + b, 0.0)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(P, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); ax.scatter(*X.T, c=y, s=64);
```



Resolución de XOR con una composición de tres Perceptrones: dos **unidades ocultas,** h_1 y h_2, y la salida y

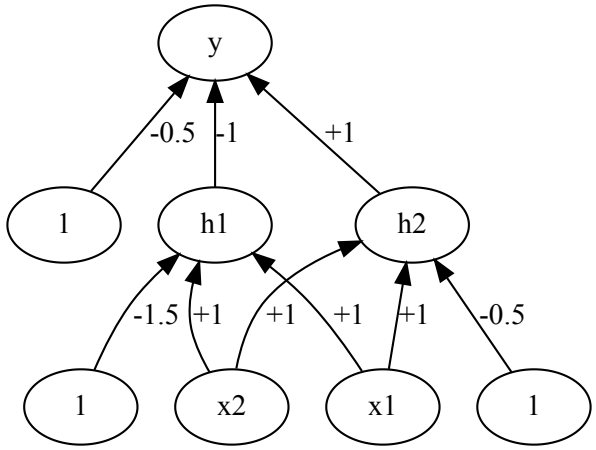
In [ ]:
```
import graphviz; graphviz.Source('''digraph { rankdir=BT
yb [label="1"] yb -> y [label="-0.5"] h1 -> y [label="-1"] h2 -> y [label="+1"]
h1b [label="1"] h1b -> h1 [label="-1.5"] x1 -> h1 [label="+1"] x2 -> h1 [label="+1"]
h2b [label="1"] h2b -> h2 [label="-0.5"] x1 -> h2 [label="+1"] x2 -> h2 [label="+1"]
}''').render(filename='MLPXOR', format='svg');
```



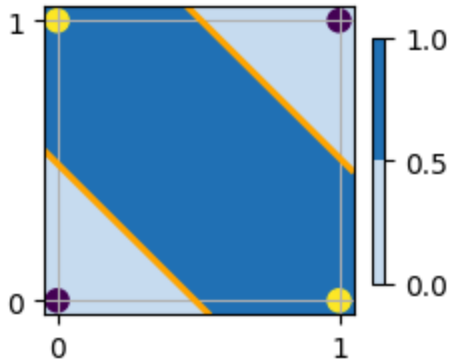
x_1	x_2	$\boldsymbol{w}_1^t\boldsymbol{x}+b_1$	h_1	$\boldsymbol{w}_2^t\boldsymbol{x}+b_2$	h_2	$\boldsymbol{w}_y^t\boldsymbol{h}+b_y$	y
0	0	-1.5	0	-0.5	0	-0.5	0
0	1	-0.5	0	$+0.5$	1	$+0.5$	1
1	0	-0.5	0	$+0.5$	1	$+0.5$	1
1	1	$+0.5$	1	$+1.5$	1	-0.5	0

$$y = \neg h_1 \wedge h_2 = \neg(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$$

Resolución de XOR con una composición de tres Perceptrones (cont.):



In [ ]:
```
w1 = np.array([1, 1]); b1 = -1.5; w2 = np.array([1, 1]); b2 = -0.5; wy = np.array([-1, 1]); by = -0.5
P = lambda x, w, b: np.heaviside(w @ x + b, 0.0)
MLPXOR = lambda x: P(np.array([P(x, w1, b1), P(x, w2, b2)]), wy, by)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(MLPXOR, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); ax.scatter(*X.T, c=y, s=64);
```



3 Perceptrones multicapa

Multilayer perceptron (MLP): composición de L funciones o **capas** $f^{(\ell)}(\boldsymbol{x}^{(\ell)};\boldsymbol{\theta}^{(\ell)})$

$$f(\boldsymbol{x};\boldsymbol{\theta}) = f^{(L)}(f^{(L-1)}(\cdots f^{(1)}(\boldsymbol{x})\cdots)) \quad \text{con} \quad \boldsymbol{x}^{(\ell+1)} = f^{(\ell)}(\boldsymbol{x}^{(\ell)}) = \boldsymbol{\varphi}^{(\ell)}(\boldsymbol{a}^{(\ell)}) \quad \text{y} \quad \boldsymbol{a}^{(\ell)} = \mathbf{W}^{(\ell)}\boldsymbol{x}^{(\ell)} + \boldsymbol{b}^{(\ell)}$$

Preactivación de la capa ℓ, $\boldsymbol{a}_\ell$: afinidad de la salida de la capa anterior, $\boldsymbol{x}^{(\ell)}$; con $\boldsymbol{x}^{(1)} = \boldsymbol{x}$

Activación/salida de la capa ℓ, $\boldsymbol{x}^{(\ell+1)}$: transformación (elemental) de $\boldsymbol{a}^{(\ell)}$ con $\varphi : \mathbb{R} \to \mathbb{R}$ **diferenciable** (y no lineal)

Terminología usual:

- **"Capa" de entrada:** $\boldsymbol{x}^{(1)} = \boldsymbol{x}$
- **Capas ocultas:** $\boldsymbol{x}^{(\ell+1)}$, $1 \leq \ell < L$, cuya dimensión indica el número de salidas o **unidades ocultas** de la capa
- **Capa de salida:** $\boldsymbol{x}^{(L+1)}$, típicamente con activación sigmoide o softmax para clasificación; identidad para regresión

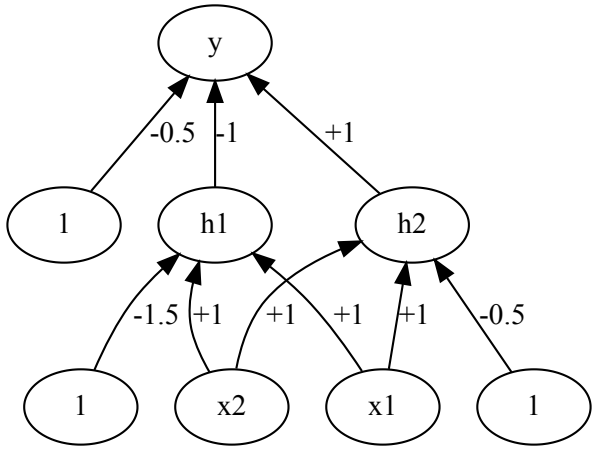
Redes superficiales (sin capas ocultas): si $L = 1$, se reducen a modelos lineales convencionales

- **Regresión logística binaria:** $f(\boldsymbol{x};\boldsymbol{\theta}) = x^{(2)},\ x^{(2)} = \sigma(a^{(1)}),\ a^{(1)} = \mathbf{W}^{(1)}\boldsymbol{x} + b^{(1)}$
- **Regresión logística:** $f(\boldsymbol{x};\boldsymbol{\theta}) = \boldsymbol{x}^{(2)},\ \boldsymbol{x}^{(2)} = \mathcal{S}(\boldsymbol{a}^{(1)}),\ \boldsymbol{a}^{(1)} = \mathbf{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$
- **Regresión lineal:** $f(\boldsymbol{x};\boldsymbol{\theta}) = x^{(2)},\ x^{(2)} = a^{(1)},\ a^{(1)} = \mathbf{W}^{(1)}\boldsymbol{x} + b^{(1)}$

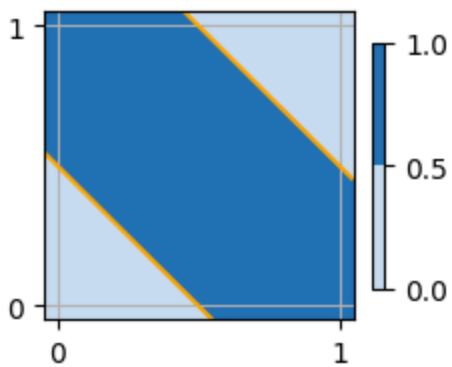
Redes profundas (con capas ocultas): si $L > 1$, son modelos lineales con **ingeniería de características** integrada

Ejercicio: sustituye las Heavisides del MLP para XOR por $\sigma(z) = \dfrac{1}{1+e^{-Tz}}$, $T = 10$

Solución: MLP para XOR con sigmoides atemperadas



In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from scipy.special import expit
w1 = np.array([1, 1]); b1 = -1.5; w2 = np.array([1, 1]); b2 = -0.5; wy = np.array([-1, 1]); by = -0.5
P = lambda x, w, b: expit(10 * (w @ x + b)); MLPXOR = lambda x: P(np.array([P(x, w1, b1), P(x, w2, b2)]), wy, by)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(MLPXOR, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); # ax.scatter(*X.T, c=y, s=64);
```



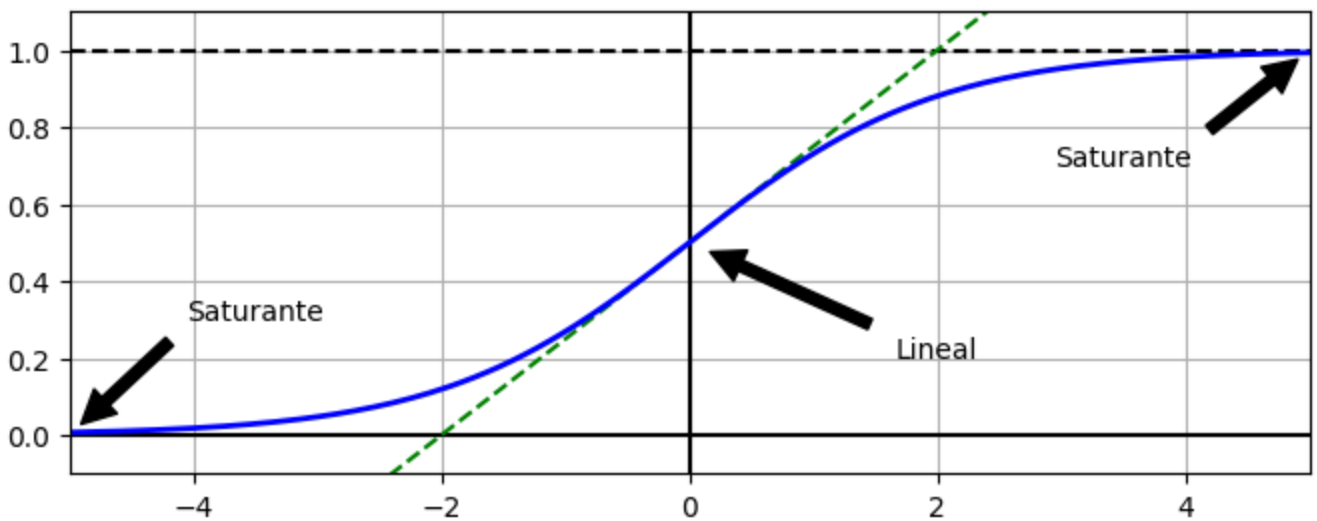
4 Funciones de activación

Necesidad de activaciones no lineales: si $\boldsymbol{\varphi}^{(\ell)}(\boldsymbol{a}^{(\ell)}) = c^{(\ell)}\boldsymbol{a}^{(\ell)}$, la red se reduce a un modelo lineal

$$\boldsymbol{a}^{(L)} = \mathbf{W}^{(L)}(c^{(L-1)}\mathbf{W}^{(L-1)}(\cdots(c^{(1)}\mathbf{W}^{(1)}\boldsymbol{x})\cdots)) = \tilde{\mathbf{W}}\boldsymbol{x} \quad \text{con} \quad \tilde{\mathbf{W}} = \mathbf{W}^{(L)}(c^{(L-1)}\mathbf{W}^{(L-1)}\cdots c^{(1)}\mathbf{W}^{(1)})$$

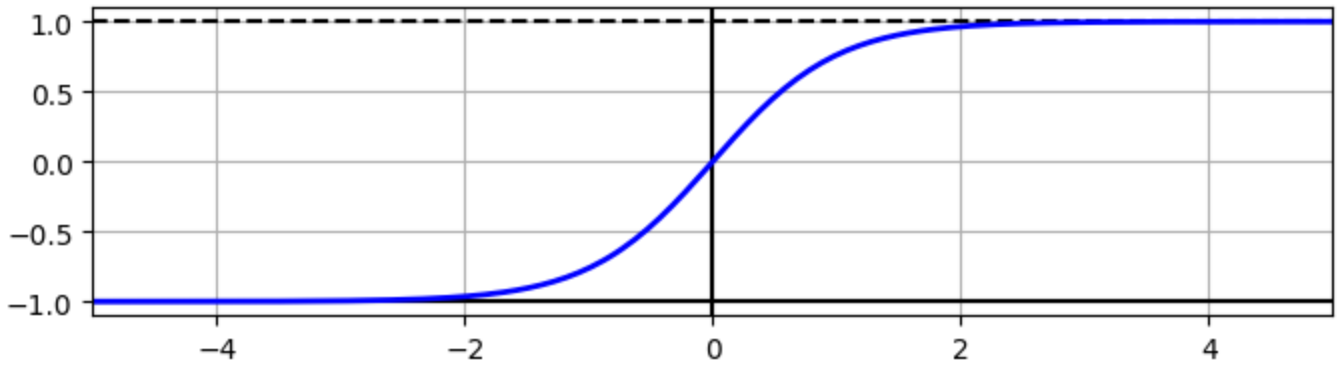
Sigmoide: muy usada en el pasado, satura a $(0, 1)$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,3)); plt.grid(True); plt.axis([-5, 5, -0.1, 1.1]);
plt.plot([-5, 5], [0, 0], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-0.1, 1.1], 'k-')
plt.plot([-5, 5], [-3/4, 7/4], 'g--'); plt.plot(a, 1 / (1 + np.exp(-a)), "b-", linewidth=2);
props = dict(facecolor='black', shrink=0.1)
plt.annotate('Saturante', xytext=(3.5, 0.7), xy=(5, 1), arrowprops=props, ha="center")
plt.annotate('Saturante', xytext=(-3.5, 0.3), xy=(-5, 0), arrowprops=props, ha="center")
plt.annotate('Lineal', xytext=(2, 0.2), xy=(0, 0.5), arrowprops=props, ha="center");
```



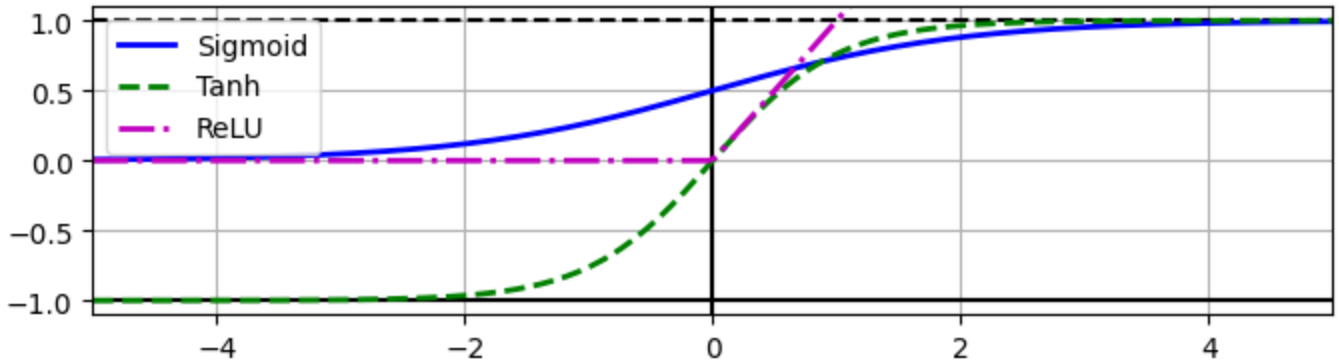
Tangente hiperbólica: muy usada en el pasado, satura a $(-1, 1)$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,2)); plt.grid(True); plt.axis([-5, 5, -1.1, 1.1]);
plt.plot([-5, 5], [-1, -1], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-1.1, 1.1], 'k-')
plt.plot(a, np.tanh(a), "b-", linewidth=2);
```



Rectified linear unit (ReLU): muy usada actualmente, $\mathrm{ReLU}(a) = \max(a, 0) = a\,\mathbb{I}(a > 0)$

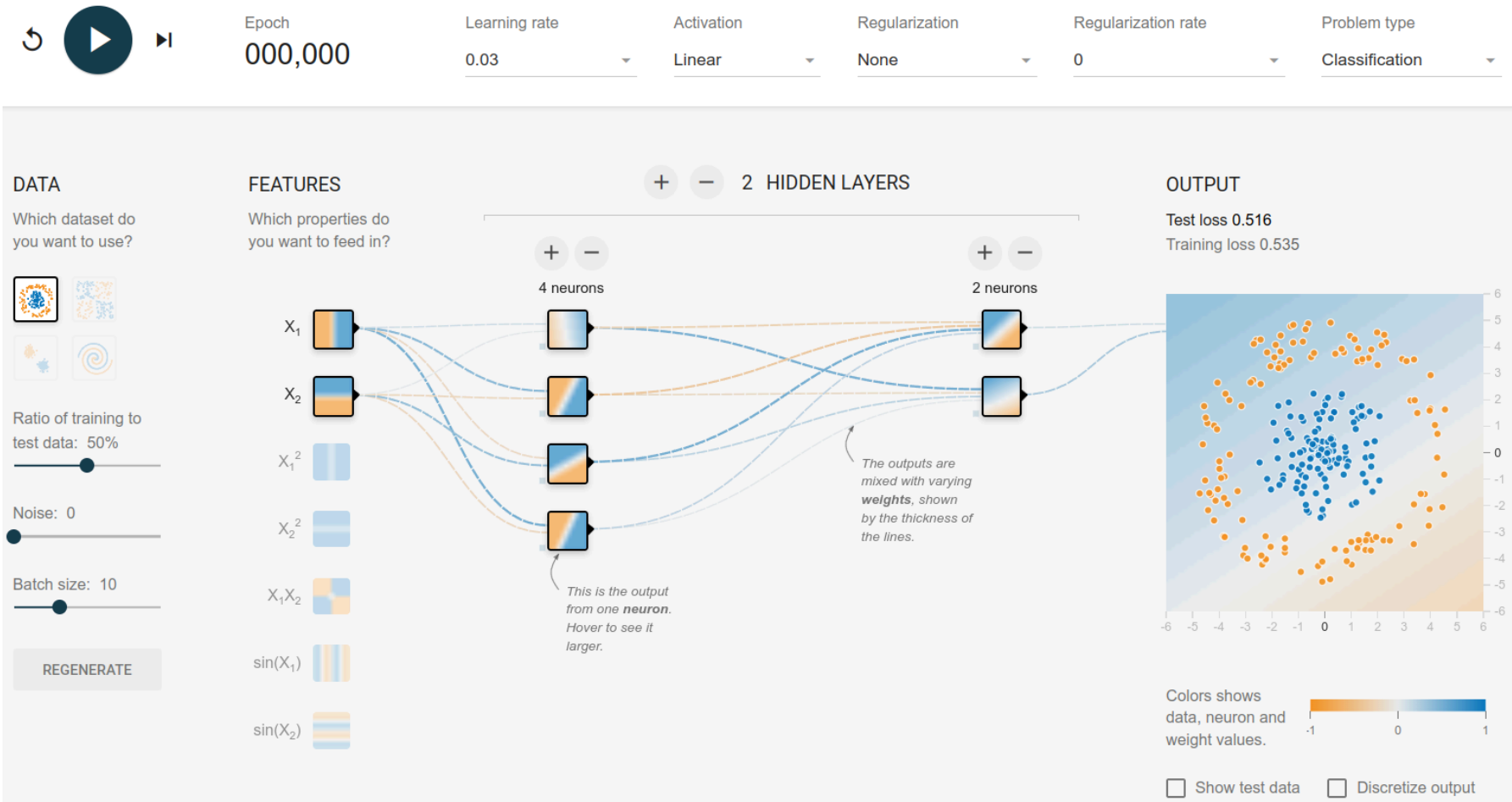
In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,2)); plt.grid(True); plt.axis([-5, 5, -1.1, 1.1]);
plt.plot([-5, 5], [-1, -1], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-1.1, 1.1], 'k-')
plt.plot(a, 1 / (1 + np.exp(-a)), "b-", linewidth=2, label="Sigmoid")
plt.plot(a, np.tanh(a), "g--", linewidth=2, label="Tanh")
plt.plot(a, np.maximum(a, 0), "m-.", linewidth=2, label="ReLU"); plt.legend(loc="upper left");
```



5 TensorFlow Playground

TensorFlow Playground: herramienta web para jugar con redes sencillas aplicadas a datos 2d

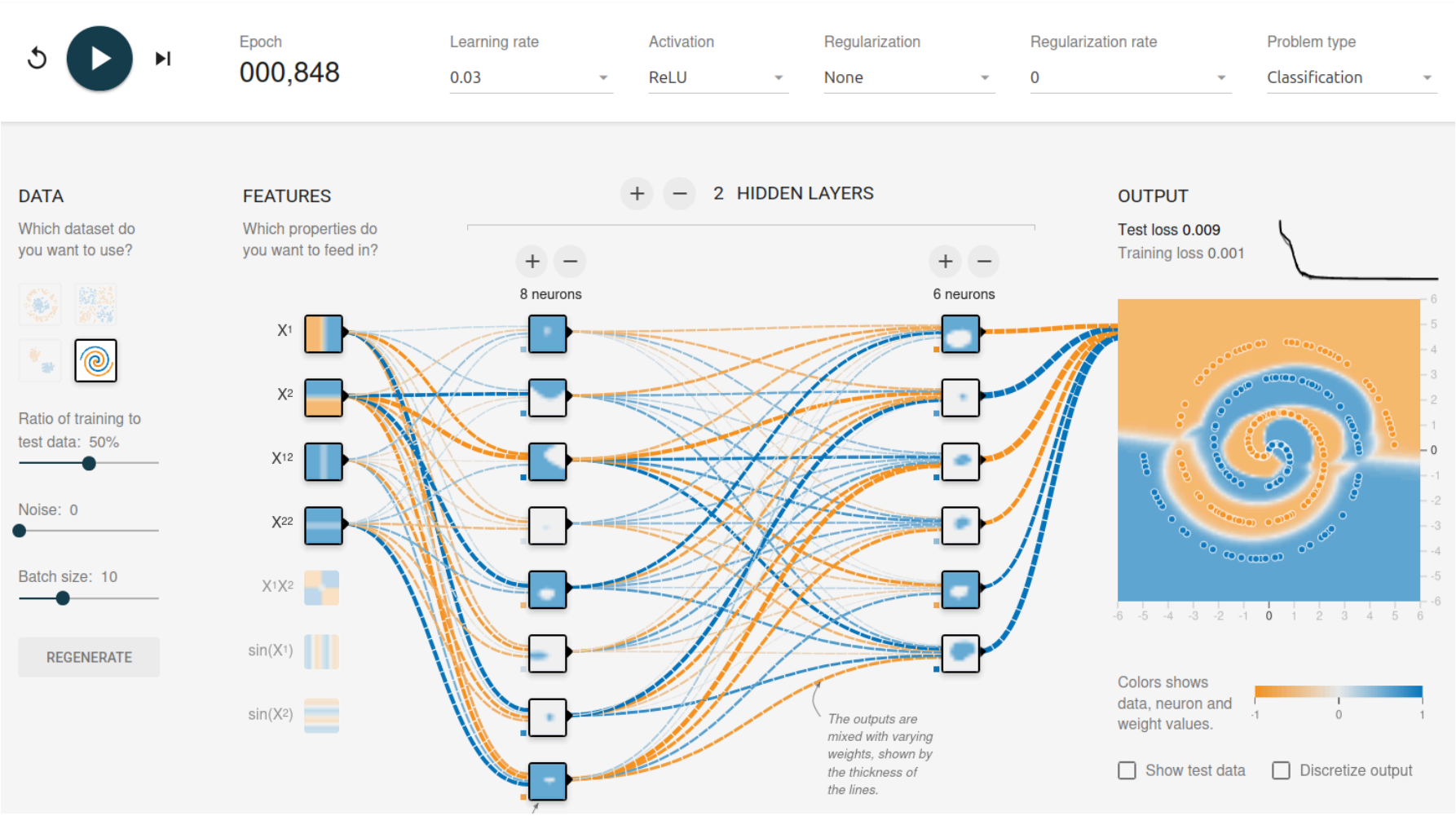
URL y github: http://playground.tensorflow.org https://github.com/tensorflow/playground



Ejercicio: por cada tarea de clasificación, halla una red minimalista que la resuelva

Solución:

- La lineal no requiere capas ocultas
- La círculo-anillo tampoco con características cuadráticas; eso, o una capa oculta de tres unidades
- La de cuadrantes se resuelve con una capa oculta de cuatro unidades
- La de espirales puede resolverse con características cuadráticas y dos capas ocultas de ocho y seis unidades



6 Ejemplo de MLP para clasificación

MLP para MNIST: entrada de 784 dimensiones, capa oculta de 800 ReLUs y salida logits para 10 clases

In [ ]:
```
import numpy as np; import datasets
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy"); ds
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
```

In [ ]:
```
import torch; from torch import nn
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
class NeuralNetwork(nn.Module):
    def __init__(self, input_shape, num_ReLUs=800):
        super().__init__()
        self.MLP = nn.Sequential(nn.Linear(input_shape, num_ReLUs), nn.ReLU(),
            nn.Linear(num_ReLUs, num_ReLUs), nn.ReLU(), nn.Linear(num_ReLUs, 10))
    def forward(self, x):
        return self.MLP(x)
model = NeuralNetwork(X_train.shape[1]).to(device)
```

In [ ]:
```
X = torch.Tensor(X_train).to(device); y = torch.Tensor(y_train).to(device, dtype=torch.long)
loss_fn = nn.CrossEntropyLoss(); optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
model.train(); epochs = 200
for t in range(epochs):
    pred = model(X); loss = loss_fn(pred, y); loss.backward(); optimizer.step(); optimizer.zero_grad()
```

In [ ]:
```
X = torch.Tensor(X_test).to(device); pred = model(X).detach().cpu().numpy()
acc = (pred.argmax(1) == y_test).sum().item() / y_test.size
print(f'Precisión {acc:.2%}')
```

```
Precisión 97.61%
```

7 Ejemplo de MLP para regresión

auto-mpg: predicción de consumo en miles per gallon

In [ ]:
```
import numpy as np; import datasets; ds = datasets.load_dataset("scikit-learn/auto-mpg"); ds
```

Out[ ]:
```
DatasetDict({
    train: Dataset({
        features: ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year', 'origi
n', 'car name'],
        num_rows: 398
    })
})
```

Formato: conversión a `dataframe` de `pandas`

In [ ]:
```
import pandas as pd; df = ds['train'].to_pandas(); df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car name      398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
```

Preproceso: tras examinar variables en el Dataset Viewer

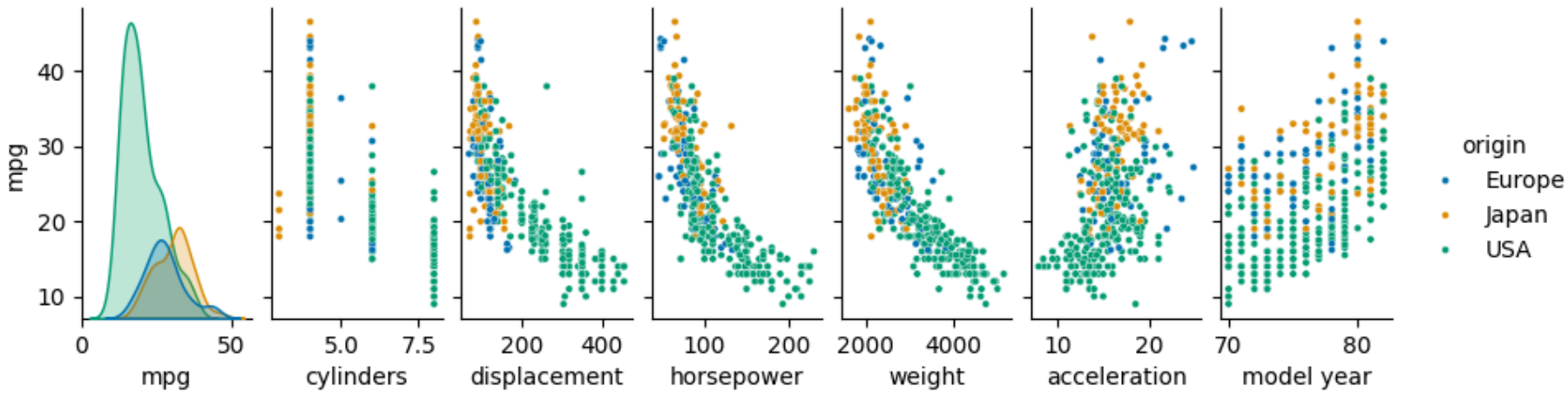
- `horsepower`: normalización de tipo tras eliminar seis casos con `"?"`
- `origin`: normalización de tipo ya que es una categórica con tres posibles valores; 1, 2 o 3
- `car name`: eliminación ya que el modelo de coche no interesa como variable explicativa

In [ ]:
```
df.drop(df[df.horsepower == "?"].index, inplace=True)
df = df.astype({'horsepower' : np.int64})
df['origin'] = df['origin'].map({1: 'USA', 2: 'Europe', 3: 'Japan'})
df = df.astype({'origin' : 'category'})
df.drop('car name', axis=1, inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           392 non-null    float64
 1   cylinders     392 non-null    int64
 2   displacement  392 non-null    float64
 3   horsepower    392 non-null    int64
 4   weight        392 non-null    int64
 5   acceleration  392 non-null    float64
 6   model year    392 non-null    int64
 7   origin        392 non-null    category
dtypes: category(1), float64(3), int64(4)
memory usage: 25.0 KB
```

Inspección visual:

In [ ]:
```
import seaborn as sns
sns.pairplot(df, y_vars='mpg', hue="origin", aspect=.5, palette='colorblind', plot_kws=dict(marker="o", s=10));
```



Diseño experimental:

- Binarización de `origin` para facilitar su tratamiento
- Partición de los datos en entrenamiento y test
- Estandarización de características

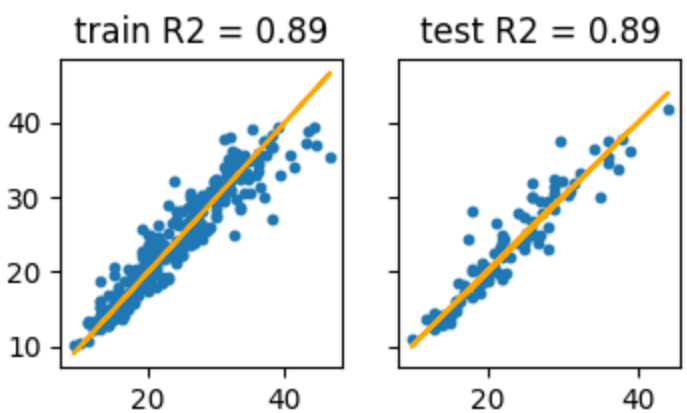
In [ ]:
```
from sklearn.preprocessing import StandardScaler; from sklearn.model_selection import train_test_split
df = pd.concat([df, pd.get_dummies(df.origin, prefix='', prefix_sep='')], axis=1)
df.drop('origin', axis=1, inplace=True); y = df.pop('mpg').to_numpy().reshape(-1, 1); X = df
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
scaler = StandardScaler(); X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test);
```

Experimento con un MLP: dos capas ocultas de 128 ReLUs y capa de salida con una unidad lineal

In [ ]:
```
import torch; from torch import nn; num_ReLUs = 128
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
class NeuralNetwork(nn.Module):
    def __init__(self, input_shape):
        super().__init__()
        self.MLP = nn.Sequential(nn.Linear(input_shape, num_ReLUs), nn.ReLU(),
            nn.Linear(num_ReLUs, num_ReLUs), nn.ReLU(), nn.Linear(num_ReLUs, 1))
    def forward(self, x):
        return self.MLP(x)
model = NeuralNetwork(X_train.shape[1]).to(device)
```

In [ ]:
```
X = torch.Tensor(X_train).to(device); y = torch.Tensor(y_train).to(device)
loss_fn = nn.MSELoss(); optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
model.train(); epochs = 500
for t in range(epochs):
    pred = model(X); loss = loss_fn(pred, y); loss.backward(); optimizer.step(); optimizer.zero_grad()
```

In [ ]:
```
import matplotlib.pyplot as plt; from sklearn.metrics import r2_score; model.eval()
y_train_pred =  model(X).detach().cpu(); y_test_pred = model(torch.Tensor(X_test).to(device)).detach().cpu()
r2_train = r2_score(y_train, y_train_pred); r2_test = r2_score(y_test, y_test_pred)
fig, axs = plt.subplots(1, 2, figsize=(4, 2), sharey = 'all')
ax = axs[0]; ax.set_title(f'train R2 = {r2_train:.2f}')
ax.scatter(y_train, y_train_pred, s=10); ax.plot(y_train, y_train, color='orange')
ax = axs[1]; ax.set_title(f'test R2 = {r2_test:.2f}')
ax.scatter(y_test, y_test_pred, s=10); ax.plot(y_test, y_test, color='orange');
```



8 Consideraciones finales

Antecedentes de las redes: las ideas básicas tras las redes son conocidas durante varias décadas (desde la cibernética)

Conexiones con la biología: muchos autores se han inspirado en la biología

Gran popularidad a partir de los 2010: por la mejora de resultados en reconocimiento automático del habla (2011) y clasificación de imágenes (2012)

GPUs y grandes conjuntos de datos supervisados: son los principales factores tras la mejora de resultados

Librerías de código abierto: facilitan la aplicación de las redes a multitud de problemas

- **Tensorflow (Google):** https://www.tensorflow.org/
- **PyTorch (Meta):** https://pytorch.org/

No necesidad de más de una capa oculta: un MLP de **una sola capa oculta** con suficientes unidades es una **aproximador de funciones universal** ya que puede aproximar toda función (mínimamente suave) tanto como se quiera

Importancia de la profundidad: las redes profundas suelen ofrecer mejores resultados ya que las últimas capas aprovechan las características aprendidas en las primeras capas